

Week 2: Linear models and causal inference

Categories and curves

Workspace setup:

```
library(tidyverse)
library(cowplot)
library(brms)
library(tidybayes)
library(patchwork)
library(here)
```

As we develop more useful models, we'll begin to practice the art of generating models with multiple estimands. An *estimand* is a quantity we want to estimate from the data. Our models may not themselves produce the answer to our central question, so we need to know how to calculate these values from the posterior distributions.

This is going to be different from prior regression courses (PSY 612), where our models were often designed to give us precisely what we wanted. For example, consider the regression:

$$\hat{Y} = b_0 + b_1(D)$$

Where Y is a continuous outcome and D is a dummy coded variable (0 = control; 1 = treatment).

- What does b_0 represent?
 - What does b_1 represent?
 - How would you calculate or estimate the means of both groups from this model?
-

Categories

If you were to fit the dummy variable model in R, your formula would be:

```
y ~ 1 + D
```

or

```
y ~ D # intercept is implied
```

Forget dummy codes. From here on out, we will incorporate categorical causes into our models by using index variables. An **index variable** contains integers that correspond to different categories. The numbers have no inherent meaning – rather, they stand as placeholders or shorthand for categories.

To fit these models in R, we're going to force the model to drop the intercept.

```
y ~ 0 + I
```

```
1 data("Howell1", package = "rethinking")
2 d <- Howell1
3 library(measurements)
4 d$height <- conv_unit(d$height, from = "cm", to = "feet")
5 d$weight <- conv_unit(d$weight, from = "kg", to = "lbs")
6 d <- d[d$age >= 18, ]
7 d$sex <- ifelse(d$male == 1, 2, 1) # 1 = female, 2 = male
8 d$sex <- factor(d$sex)
9 head(d[, c("male", "sex")])
```

	male	sex
1	1	2
2	0	1
3	0	1
4	1	2
5	0	1
6	1	2

Mathematical model

Let's write a mathematical model to express weight in terms of sex.

$$\begin{aligned}w_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \alpha_{SEX[i]} \\ \alpha_j &\sim \text{Normal}(130, 20) \text{ for } j = 1..2 \\ \sigma &\sim \text{Uniform}(0, 25)\end{aligned}$$

Fitting the model using brms()

```
m1 <- brm(  
  data = d,  
  family = gaussian,  
  bf(weight ~ 0 + a,  
    a ~ 0 + sex,  
    nl = TRUE),  
  prior = c(prior( normal(130, 20), class=b, nlpar = a),  
    prior( uniform(0, 25), class=sigma, ub=25)),  
  iter = 2000, warmup = 1000, seed = 3, chains=1,  
  file = here("files/models/22.1")  
)
```

- ① A few things to note: first is that we've wrapped our formula in the function `bf()`, which allows us to build a model with multiple formulas. Cool. Next, we've forced the model to drop the intercept by using 0 instead of 1. Finally, we've created a parameter `a` which is a constant, but we're going to write another formula to determine `a` in the next line. By the way, you can call this anything you want. Try using `mean` or anything that will make sense to you. Just make sure you replace `a` everywhere else in the code.
- ② Defining `a`. Again, we drop the intercept, and now `a` is determined by the index variable or group.
- ③ This simply allows us to use non-linear language to fit our models, even though this is a linear model.
- ④ Once you put yourself in non-linear land, you have to specify which parameter each prior is for.

```
posterior_summary(m1)
```

	Estimate	Est.Error	Q2.5	Q97.5
b_a_sex1	92.27947	0.91187785	90.50115	94.05188
b_a_sex2	107.22803	0.91554891	105.57421	109.04851
sigma	12.19829	0.46172983	11.34413	13.13513
lprior	-13.47705	0.09932469	-13.66427	-13.27516
lp__	-1390.30623	1.23903576	-1393.80224	-1388.90647

Here, we are given the estimates of the parameters specified in our model: the average weight of women (b_a_sex1) and the average weight of men (b_a_sex2). But our question is whether these average weights are different. How do we get that?

```
post = as_draws_df(m1)
head(post)
```

```
# A draws_df: 6 iterations, 1 chains, and 5 variables
  b_a_sex1 b_a_sex2 sigma lprior  lp__
1      92      106    13    -14 -1391
2      92      109    13    -13 -1391
3      91      109    12    -13 -1392
4      92      109    12    -13 -1391
5      92      109    12    -13 -1391
6      91      106    13    -14 -1391
# ... hidden reserved variables {'.chain', '.iteration', '.draw'}
```

```
post %>%
  mutate(diff_fm = b_a_sex1 - b_a_sex2) %>%
  pivot_longer(cols = c(b_a_sex1:sigma, diff_fm)) %>%
  group_by(name) %>%
  mean_qi(value, .width = .89)
```

Warning: Dropping 'draws_df' class as required metadata was removed.

```
# A tibble: 4 x 7
  name      value .lower .upper .width .point .interval
<chr>    <dbl> <dbl> <dbl> <dbl> <chr> <chr>
```

1	b_a_sex1	92.3	90.9	93.8	0.89	mean	qi
2	b_a_sex2	107.	106.	109.	0.89	mean	qi
3	diff_fm	-14.9	-17.0	-12.9	0.89	mean	qi
4	sigma	12.2	11.5	12.9	0.89	mean	qi

Calculate the contrast

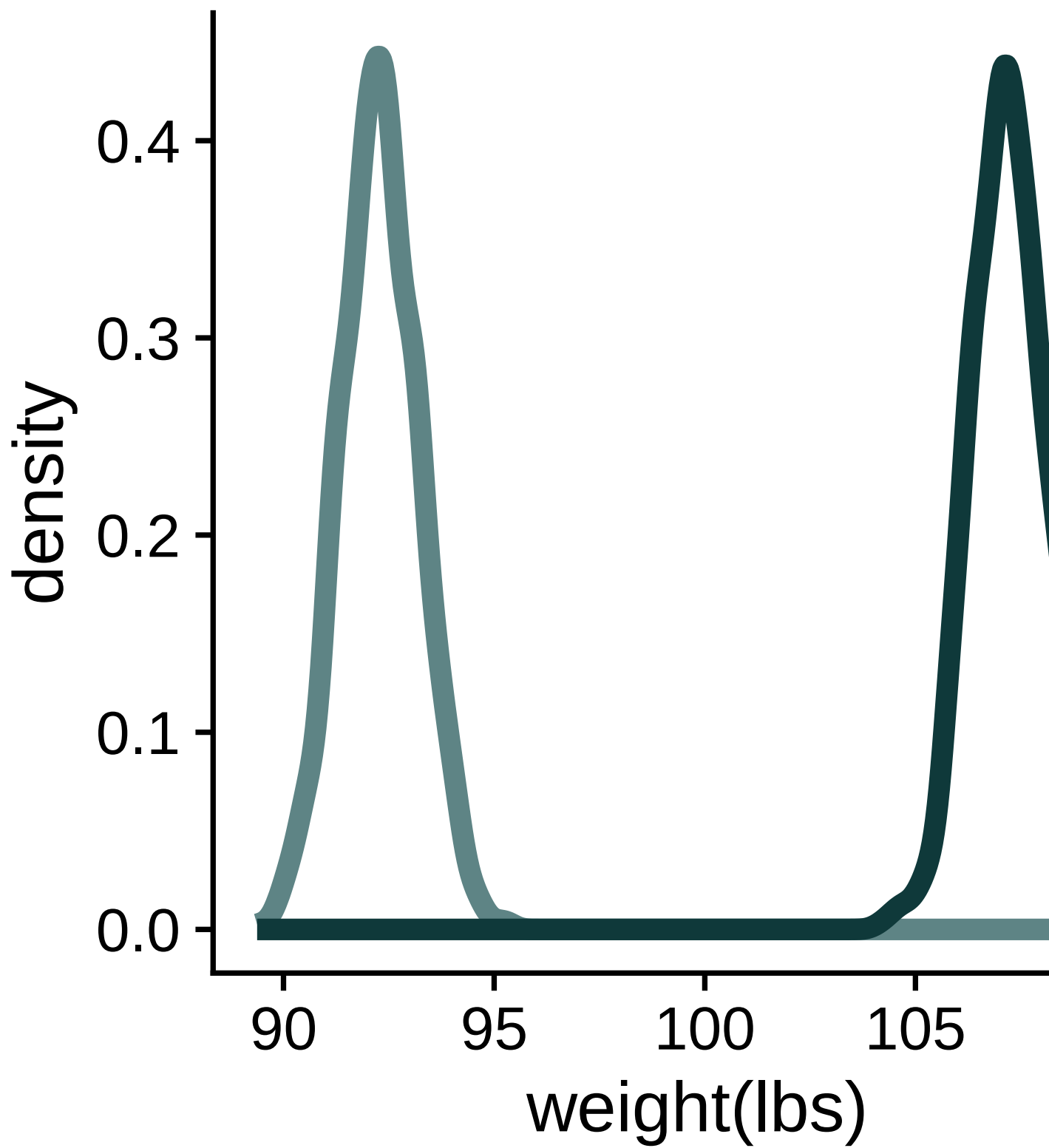
We can create two plots. One is the posterior distributions of average female and male weights and one is the average difference.

```
p1 <- post %>%
  pivot_longer(starts_with("b")) %>%
  mutate(sex = ifelse(str_detect(name, "1"), "female", "male")) %>%
  ggplot(aes(x=value, color = sex)) +
  geom_density(linewidth = 2) +
  labs(x = "weight(lbs)")

p2 <- post %>%
  mutate(diff_fm = b_a_sex1 - b_a_sex2) %>%
  ggplot(aes(x=diff_fm)) +
  geom_density(linewidth = 2) +
  labs(x = "difference in weight(lbs)")

(p1 | p2)
```

Warning: Dropping 'draws_df' class as required metadata was removed.



Expected values vs predicted values

A note that the distributions of the *mean* weights is not the same as the distribution of weights period. For that, we need the posterior predictive distributions. Here are two methods for getting predicted values.

Method 1: simulate using the `rnorm()` function. This is more intuitive and will help you mentally reconnect the parameters of your statistical model with the causal model you started with. But there's more room for human error.

```
pred_f <- rnorm(nrow(post), mean = post$b_a_sex1, sd = post$sigma )
pred_m <- rnorm(nrow(post), mean = post$b_a_sex2, sd = post$sigma )

pred_post = data.frame(pred_f, pred_m) %>%
  mutate(diff = pred_f-pred_m)
head(pred_post)
```

	pred_f	pred_m	diff
1	94.30237	69.6129	24.68947
2	93.30000	121.5658	-28.26577
3	94.81082	125.6591	-30.84826
4	106.38550	145.6095	-39.22399
5	90.91949	114.1132	-23.19373
6	100.56856	112.1951	-11.62655

Expected values vs predicted values

Method 2: use the `predicted_draws()` function. This is less intuitive, but there's less room for you to make a mistake.

```
nd = distinct(d, sex)
pred_all = predicted_draws(object=m1, newdata=nd) %>%
  ungroup %>% select(-.row) %>%
  pivot_wider(names_from = sex, names_prefix = "sex", values_from = .prediction) %>%
  mutate(diff = sex1-sex2)
head(pred_all)
```

```
# A tibble: 6 x 6
  .chain .iteration .draw sex2 sex1 diff
  <int>    <int> <int> <dbl> <dbl> <dbl>
1     NA      NA     1 102.  101. -1.58
2     NA      NA     2  96.5  88.5 -7.99
3     NA      NA     3 128.  114. -13.4
4     NA      NA     4 131.   72.2 -58.4
5     NA      NA     5 102.  105.   2.93
6     NA      NA     6 109.   71.1 -38.4
```

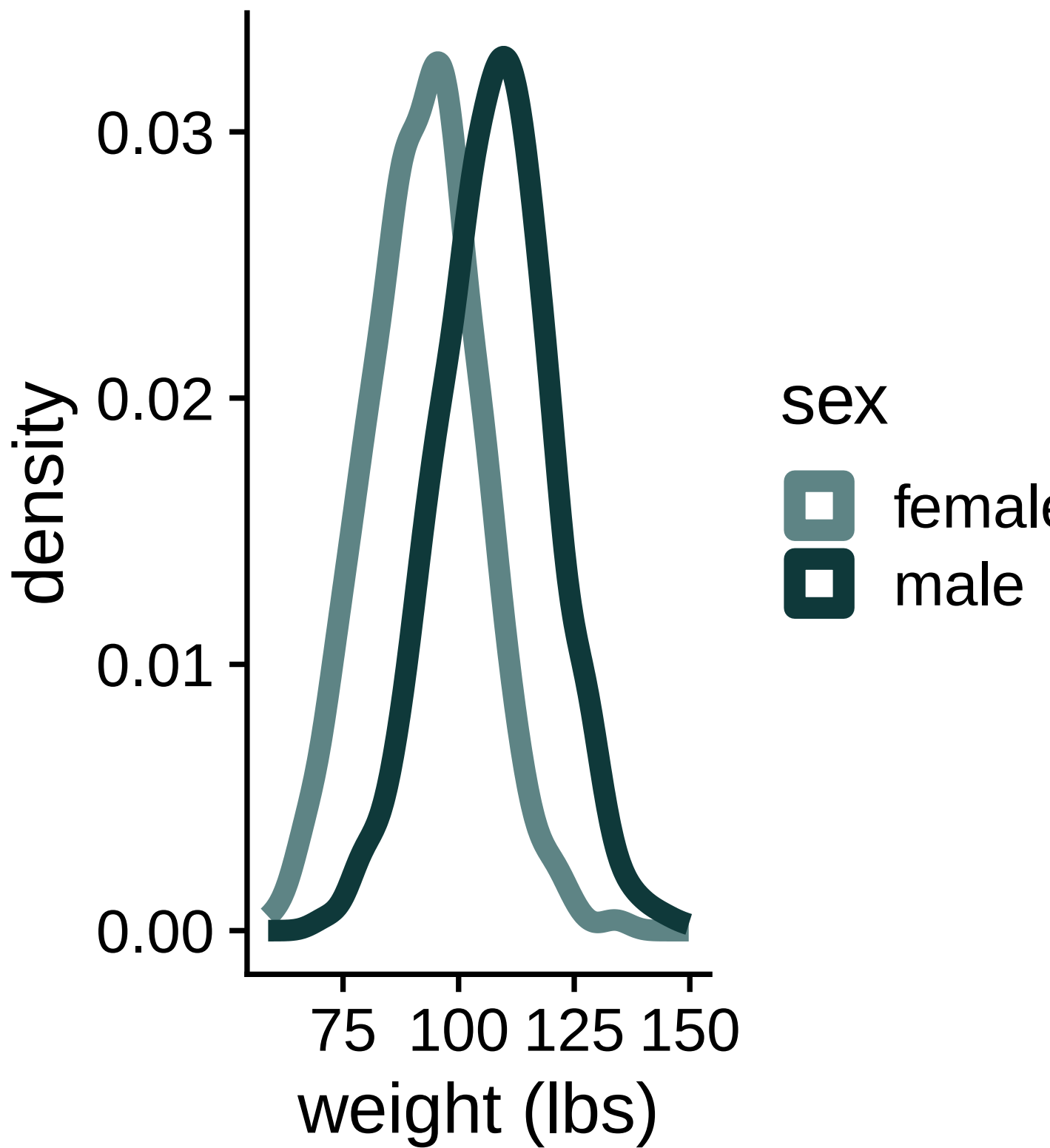
```
# plot male and female distributions using the first version
p1 <- pred_post %>% pivot_longer(starts_with("pred")) %>%
  mutate(sex = ifelse(name == "pred_f", "female", "male")) %>%
  ggplot(aes(x = value, color = sex)) +
  geom_density(linewidth = 2) +
  labs(x = "weight (lbs)")

# plot difference distribution using the second version
# Compute density first
density_data <- density(pred_all$diff)

# Convert to a tibble for plotting
density_df <- tibble(
  x = density_data$x,
  y = density_data$y,
  fill_group = ifelse(x < 0, "male", "female") # Define fill condition
)

# Plot with area fill
p2 <- ggplot(density_df, aes(x = x, y = y, fill = fill_group)) +
  geom_area() + # Adjust transparency if needed
  geom_line(linewidth = 1.2, color = "black") + # Keep one continuous curve
  labs(x = "Difference in weight (F-M)", y = "density") +
  guides(fill = "none")

(p1 | p2)
```

exercise

In the `rethinking` package, the dataset `milk` contains information about the composition of milk across primate species, as well as some other facts about those species. The taxonomic membership of each species is included in the variable `clade`; there are four categories.

1. Create variable in the dataset to assign an index value to each of the 4 categories.
2. Standardize the milk energy variable (`kcal.per.g`).¹
3. Write a mathematical model to express the average milk energy (in standardized kilocalories) in each clade.

solution

```
data("milk", package="rethinking")
str(milk)
```

```
'data.frame':  29 obs. of  8 variables:
 $ clade      : Factor w/ 4 levels "Ape","New World Monkey",...: 4 4 4 4 4 2 2 2 2 2 ...
 $ species    : Factor w/ 29 levels "A palliata","Alouatta seniculus",...: 11 8 9 10 16 2 ...
 $ kcal.per.g : num  0.49 0.51 0.46 0.48 0.6 0.47 0.56 0.89 0.91 0.92 ...
 $ perc.fat   : num  16.6 19.3 14.1 14.9 27.3 ...
 $ perc.protein : num  15.4 16.9 16.9 13.2 19.5 ...
 $ perc.lactose : num  68 63.8 69 71.9 53.2 ...
 $ mass       : num  1.95 2.09 2.51 1.62 2.19 5.25 5.37 2.51 0.71 0.68 ...
 $ neocortex.perc: num  55.2 NA NA NA NA ...
```

```
milk$clade_id <- as.integer(milk$clade)
milk$clade_id <- as.factor(milk$clade_id)
milk$K <- rethinking::standardize(milk$kcal.per.g)
```

¹You don't need to be an expert in primate biology to have a sense of what is reasonable for these values after we standardize.

$$K_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha_{\text{CLADE}[i]}$$

$$\alpha_i \sim \text{Normal}(0, 0.5) \text{ for } j = 1..4$$

$$\sigma \sim \text{Exponential}(1)$$

Exercise: Now fit your model using `brms()`. It's ok if your mathematical model is a bit different from mine.

solution

```
m2 <- brm(
  data=milk,
  family=gaussian,
  bf(K ~ 0 + a,
     a ~ 0 + clade_id,
     nl = TRUE),
  prior = c( prior(normal(0,.5), class=b, nlpar=a),
             prior(exponential(1), class=sigma)),
  iter = 2000, warmup = 1000, seed = 3, chains=1,
  file = here("files/models/22.2")
)

posterior_summary(m2)
```

	Estimate	Est.Error	Q2.5	Q97.5
b_a_clade_id1	-0.4641202	0.2256089	-0.89176596	-0.03325312
b_a_clade_id2	0.3417955	0.2276394	-0.12749964	0.78516752
b_a_clade_id3	0.6493047	0.2857904	0.09024468	1.18518255
b_a_clade_id4	-0.5387150	0.2962131	-1.10847821	0.03517569
sigma	0.7991983	0.1178092	0.61678499	1.07557053
lprior	-4.3341786	1.1496078	-6.91138553	-2.49326609
lp__	-38.8228118	1.5985564	-42.59133454	-36.64554417

exercise

Plot the following distributions:

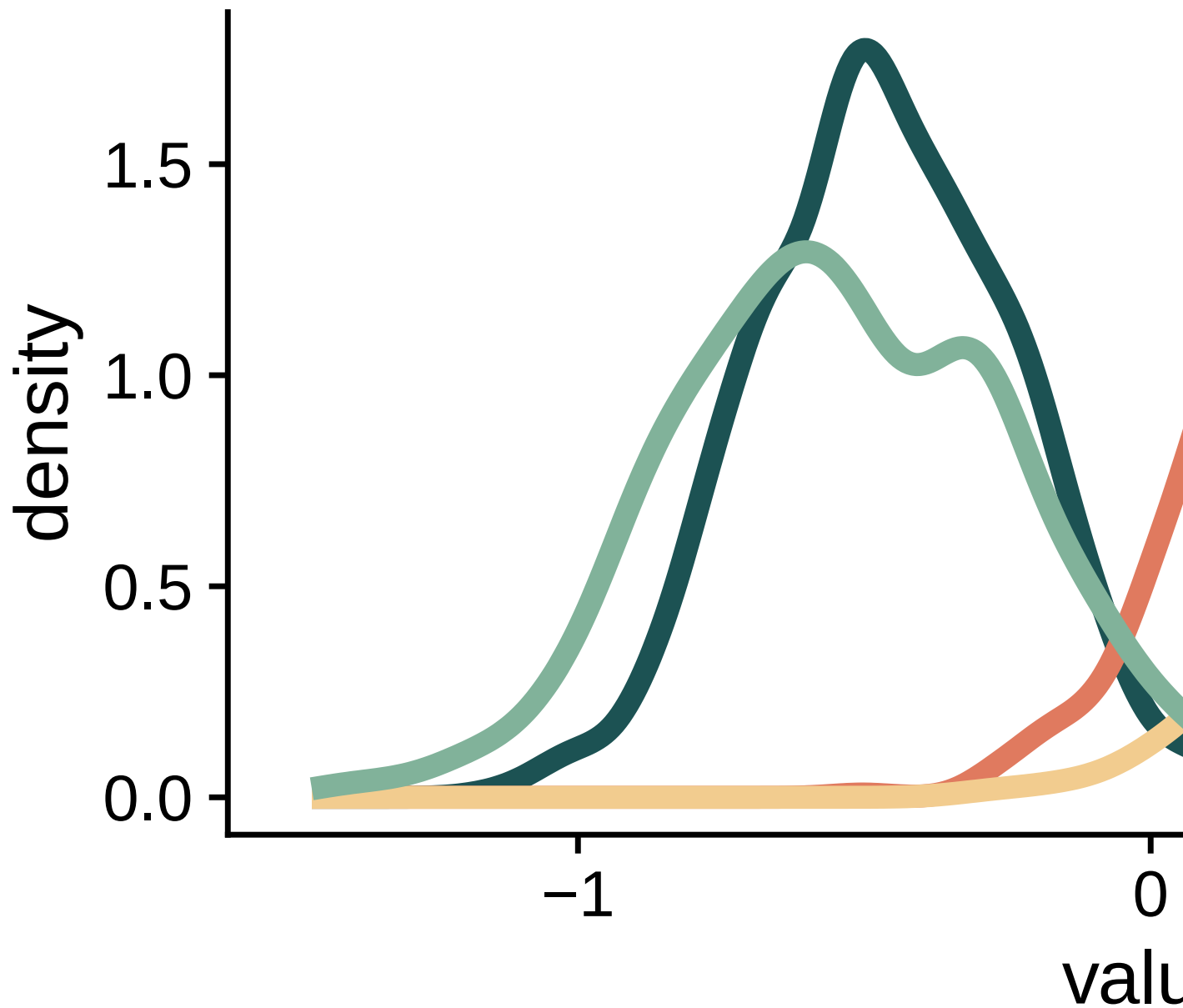
- Posterior distribution of average milk energy by clade.
 - Posterior distribution of predicted milk energy values by clade.
-

solution

```
post <- as_draws_df( m2 )
post %>%
  pivot_longer(starts_with("b")) %>%
  mutate(
    name = str_extract(name, "[0-9]"),
    name = factor(name, labels = levels(milk$clade))) %>%
  ggplot(aes(x = value, color = name)) +
  geom_density(linewidth = 2) +
  labs(title = "Posterior distribution of expected milk energy") +
  theme(legend.position = "bottom")
```

Warning: Dropping 'draws_df' class as required metadata was removed.

Posterior distribution of

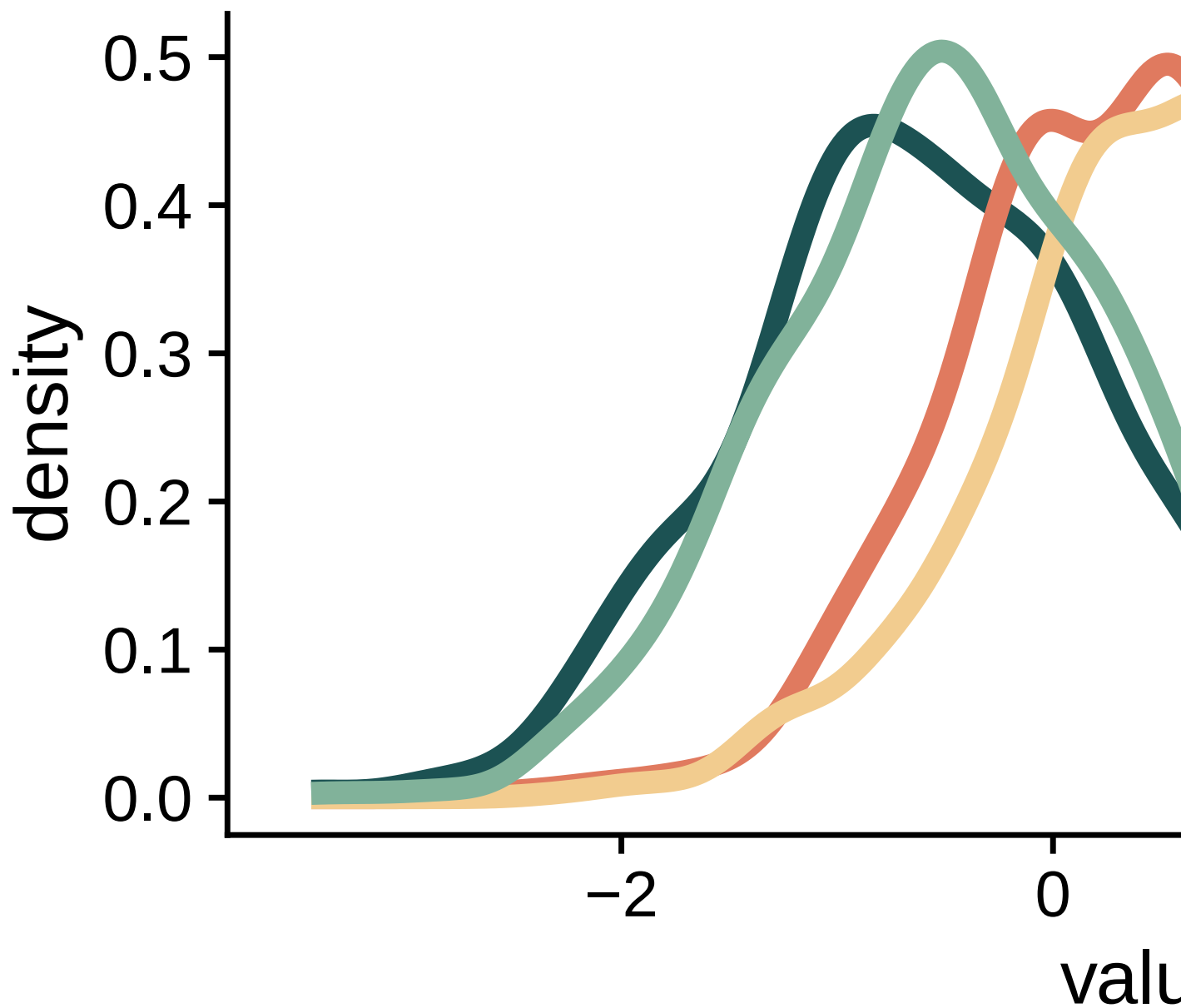


name ■ Ape ■ New World

solution

```
nd = distinct(milk, clade_id)
ppd <- posterior_predict( m2, newdata = nd)
ppd = as.data.frame(ppd)
names(ppd) = levels(milk$clade)
ppd %>%
  pivot_longer(everything()) %>%
  ggplot(aes(x = value, color = name)) +
  geom_density(linewidth = 2) +
  labs(title = "Posterior predictive distribution of predicted milk energy") +
  theme(legend.position = "bottom")
```

Posterior predictive dis

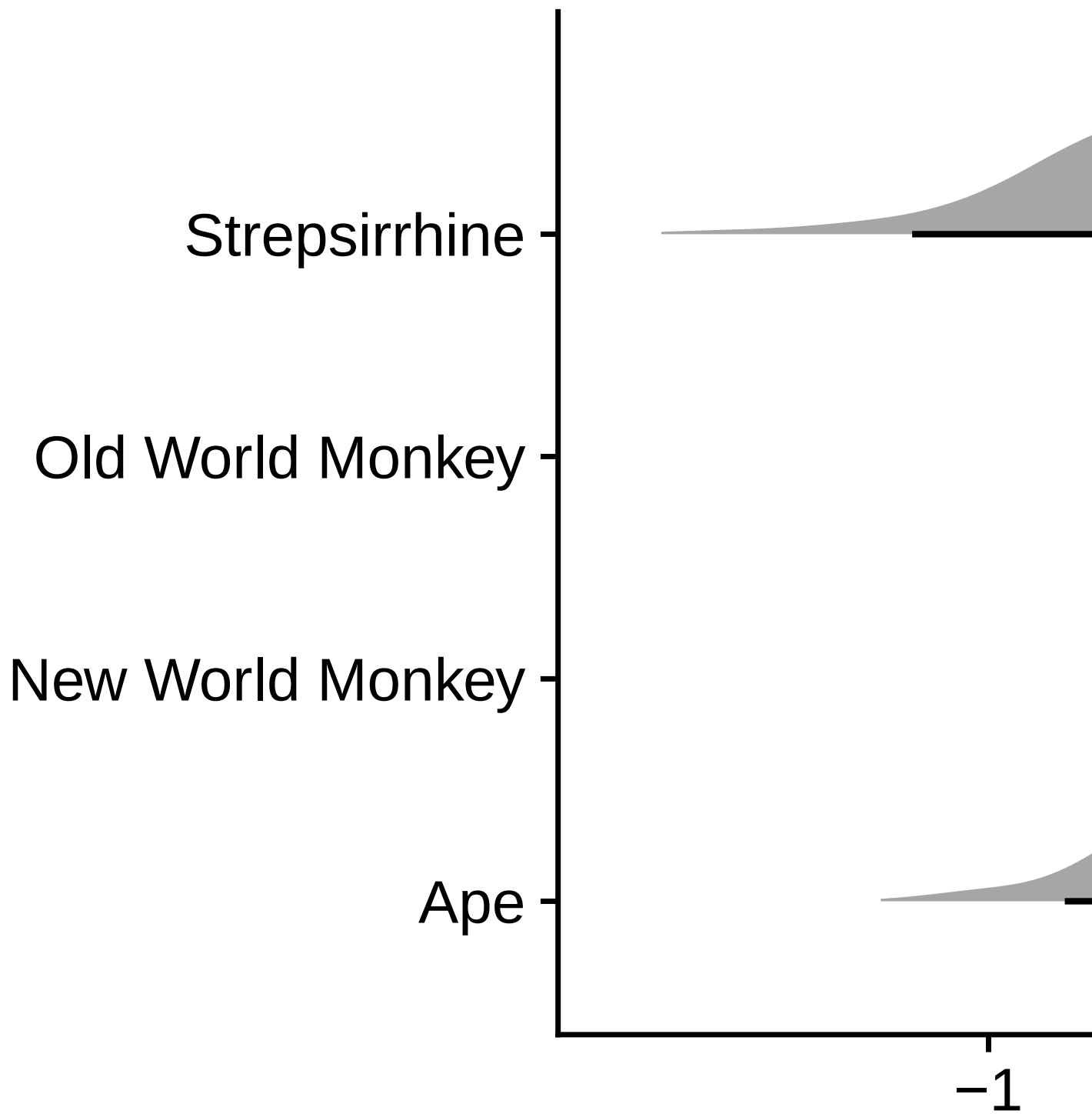


name Ape New World

Plotting with brms and tidybayes

```
as_draws_df(m2) %>%  
  pivot_longer(starts_with("b")) %>%  
  mutate(  
    clade = str_extract(name, "[0-9]"),  
    clade = as.numeric(clade),  
    clade = factor(clade, labels=levels(milk$clade))  
  ) %>%  
  ggplot(aes(y = clade, x = value)) +  
  stat_halfeye() +  
  labs(x="mean", y=NULL)
```

Warning: Dropping 'draws_df' class as required metadata was removed.



Combining index variables and slopes

Let's return to the weight example. What if we want to control for height?

$$\begin{aligned}w_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \alpha_{S[i]} + \beta_{S[i]}(H_i - \bar{H}) \\ \alpha_j &\sim \text{Normal}(130, 20) \text{ for } j = 1..2 \\ \beta_j &\sim \text{Normal}(0, 25) \text{ for } j = 1..2 \\ \sigma &\sim \text{Uniform}(0, 50)\end{aligned}$$

```
d$height_c <- d$height - mean(d$height)
m3 <- brm(
  data = d,
  family = gaussian,
  bf(weight ~ 0 + a + b*height_c,
    a ~ 0 + sex,
    b ~ 0 + sex,
    nl = TRUE),
  prior = c(prior(normal(130, 20), class = b, nlpar = a),
    prior(normal( 0, 25), class = b, nlpar = b),
    prior(uniform( 0, 50), class = sigma, lb=0, ub=50)),
  iter = 2000, warmup = 1000, seed = 3, chains=1,
  file = here("files/models/22.3")
)
```

```
posterior_summary(m3)
```

	Estimate	Est.Error	Q2.5	Q97.5
b_a_sex1	99.432780	0.9552998	97.482112	101.14593
b_a_sex2	99.540714	1.0416720	97.493362	101.51419
b_b_sex1	43.123143	4.0474656	35.486369	50.76018
b_b_sex2	40.246225	3.7280322	32.940455	47.30186
sigma	9.411125	0.3685640	8.742146	10.19855
lprior	-25.154833	0.3833938	-25.936707	-24.44280
lp__	-1310.971276	1.5505617	-1314.791684	-1308.86432

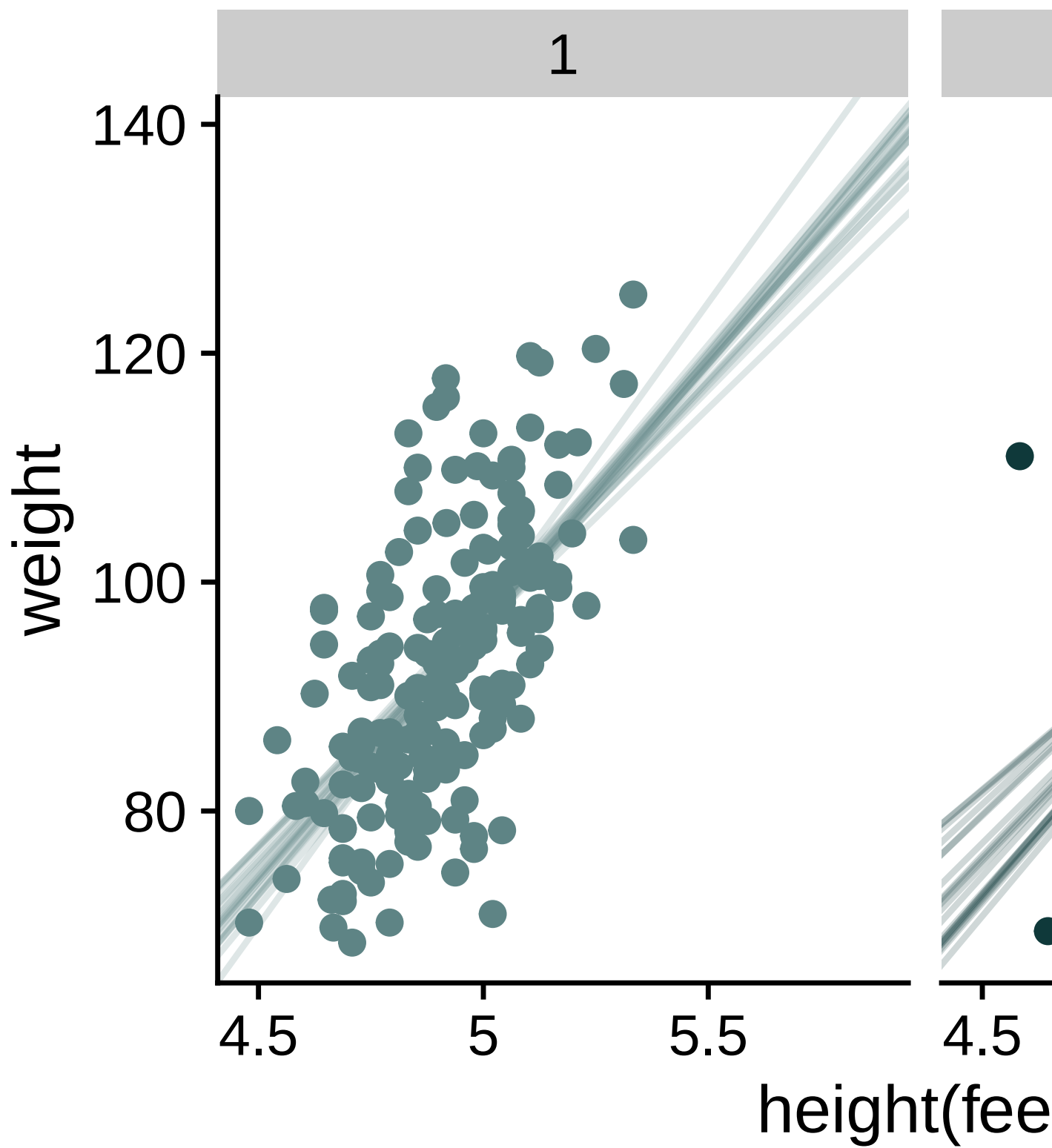
Plot the slopes from the posterior

```
post = as_draws_df(m3) %>%
  pivot_longer(starts_with("b"),
               names_to = c("parameter", "sex"),
               names_sep = 3) %>%
  mutate(sex = str_extract(sex, "[0-9]")) %>%
  pivot_wider(names_from = parameter, values_from = value)
```

Warning: Dropping 'draws_df' class as required metadata was removed.

```
xlabs = seq(4,6,by=.5)

d %>%
  ggplot(aes(x=height_c, y=weight)) +
  geom_point(aes(color=sex)) +
  geom_abline(
    aes(intercept=b_a, slope=b_b, color=sex),
    data=post[1:40, ],
    alpha=.2
  ) +
  scale_x_continuous("height(feet)",
                     breaks=xlabs-mean(d$height),
                     labels=xlabs) +
  facet_wrap(~sex)
```



exercise

Return to the `milk` data. Write a mathematical model expressing the energy of milk as a function of the species body mass (`mass`) and clade category. Be sure to include priors. Fit your model using `brms()`.

solution

$$\begin{aligned}K_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \alpha_{\text{CLADE}[i]} + \beta_{\text{CLADE}[i]}(M - \bar{M}) \\ \alpha_i &\sim \text{Normal}(0, 0.5) \text{ for } j = 1..4 \\ \beta_i &\sim \text{Normal}(0, 0.5) \text{ for } j = 1..4 \\ \sigma &\sim \text{Exponential}(1)\end{aligned}$$

```
milk$mass_c = milk$mass - mean(milk$mass)

m4 <- brm(
  data = milk,
  family = gaussian,
  bf(K ~ 0 + a + b*mass_c,
    a ~ 0 + clade_id,
    b ~ 0 + clade_id,
    nl = TRUE),
  prior = c(prior( normal(0,.5), class=b, nlpar = a),
    prior( normal(0,.5), class=b, nlpar = b),
    prior( exponential(1), class=sigma)),
  iter = 2000, warmup = 1000, seed = 3, chains=1,
  file = here("files/models/22.4")
)
```

```
posterior_summary(m4)
```

	Estimate	Est.Error	Q2.5	Q97.5
b_a_clade_id1	-0.40476845	0.283412266	-0.94455375	0.16726235
b_a_clade_id2	-0.22553592	0.492279060	-1.17806955	0.74233613
b_a_clade_id3	0.34132362	0.432395484	-0.55499585	1.17621902
b_a_clade_id4	-0.01934509	0.507290802	-1.03169378	0.97077044
b_b_clade_id1	-0.00306615	0.008338136	-0.02027642	0.01268911
b_b_clade_id2	-0.05655136	0.042402253	-0.14149982	0.03104465
b_b_clade_id3	-0.04987753	0.054476898	-0.15271035	0.05688258
b_b_clade_id4	0.06473665	0.049274690	-0.03741264	0.15757431
sigma	0.79976985	0.119804429	0.59987239	1.06135351
lprior	-4.83595273	1.420149956	-8.20037391	-2.97899290
lp__	-39.39492980	2.284703218	-45.21163214	-36.02944463

```
post = as_draws_df(m4) %>%  
  pivot_longer(starts_with("b"),  
               names_to = c("parameter", "clade_id"),  
               names_sep = 3) %>%  
  mutate(clade_id = str_extract(clade_id, "[0-9]")) %>%  
  pivot_wider(names_from = parameter, values_from = value)
```

Warning: Dropping 'draws_df' class as required metadata was removed.

```
milk %>%  
  ggplot(aes(x=mass_c, y=K)) +  
  geom_point(aes(color=clade_id)) +  
  geom_abline(  
    aes(intercept=b_a, slope=b_b, color=clade_id),  
    data=post[1:80, ],  
    alpha=.2  
  ) +  
  facet_wrap(~clade_id) +  
  guides(color="none")
```

